LAPORAN PRAKTIKUM

PEMROGRAMAN MOBILE

Pertemuan 2: Layout, ListView, dan Navigasi Antar Halaman

Nama	:	Muhamad Dika Ramadhan
NIM	:	20240801045
Program Studi	:	Teknik Informatika
Mata Kuliah	:	Pemrograman Mobile
Pertemuan	:	2

1. Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Menyusun layout dengan Container, Padding, Row, Column, dan Expanded.
2. Menampilkan daftar data dengan ListView.builder, Card, dan ListTile.
3. Memodelkan data sederhana dengan class Dart.
4. Berpindah halaman dan mengirim data memakai Navigator.push dan Navigator.pop.

2. Landasan Teori

Layout di Flutter dibangun dengan menyusun widget secara bertingkat. Widget layout menentukan ukuran, jarak, dan posisi widget anak, sedangkan ListView dipakai untuk menampilkan data berulang yang dapat di-scroll. Perpindahan antar halaman dikelola oleh Navigator yang bekerja seperti tumpukan (stack) halaman.

Widget	Fungsi
Container	Kotak serbaguna: ukuran, warna, border, radius, padding, margin.
Padding	Memberi jarak di sekeliling widget anak.
Row / Column	Menyusun anak secara horizontal / vertikal.
Expanded	Membuat anak mengisi sisa ruang pada Row/Column.
ListView.builder	Daftar yang di-render sesuai kebutuhan (efisien untuk data banyak).
Card + ListTile	Kartu berisi baris standar: ikon, judul, subjudul, ikon akhir.
Navigator	Mengelola stack halaman: push menambah, pop menutup.

Sumbu layout: pada Row, main axis adalah horizontal dan cross axis vertikal; pada Column sebaliknya. Properti mainAxisAlignment dan crossAxisAlignment mengatur perataan pada masing-masing sumbu.

3. Alat dan Bahan

- Flutter SDK, VS Code/Android Studio, dan emulator dari Pertemuan 1
- Proyek baru: flutter create praktikum_2

4. Langkah dan Hasil Praktikum

Bagian A. Layout Kartu Profil

Isi lib/main.dart diganti dengan MyApp (yang menggunakan tema Material 3 dengan warna dasar biru) dan halaman ProfilePage berikut:

```
class ProfilePage extends StatelessWidget {
  const ProfilePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Profil')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Container(
          padding: const EdgeInsets.all(16),
          decoration: BoxDecoration(
            color: Colors.blue.shade50,
            borderRadius: BorderRadius.circular(12),
          ),
          child: Row(
            children: [
              const CircleAvatar(
                radius: 32,
                child: Icon(Icons.person, size: 32),
              ),
              const SizedBox(width: 16),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: const [
                    Text('Muhamad Dika Ramadhan',
                        style: TextStyle(fontSize: 20,
                            fontWeight: FontWeight.bold)),
                    Text('NIM: 20240801045'),
                  ],
                ),
              ),
            ],
```

```
          ),
        ),
      ),
    );
  }
}
```

Bagian Kode	Penjelasan
ThemeData(colorSchemeSeed: Colors.blue, useMaterial3: true)	Membuat seluruh skema warna aplikasi dari satu warna dasar (biru) dengan gaya Material 3.
Padding(padding: EdgeInsets.all(16))	Memberi jarak 16 piksel di keempat sisi pada widget anak.
Container	Kotak pembungkus kartu profil. Memiliki padding sendiri dan decoration.
BoxDecoration	Mengatur tampilan Container: warna latar (color) dan sudut membulat (borderRadius).
Colors.blue.shade50	Varian biru yang sangat muda sebagai warna latar kartu.
BorderRadius.circular(12)	Membulatkan keempat sudut dengan radius 12.
Row	Menyusun avatar dan teks berdampingan secara horizontal.
CircleAvatar	Lingkaran untuk foto/ikon profil; radius menentukan ukurannya.
Expanded	Membuat Column teks mengisi sisa lebar Row. Tanpa Expanded, nama yang panjang akan melewati batas layar (overflow, muncul garis kuning-hitam).
CrossAxisAlignment.start	Meratakan teks ke kiri (sumbu silang pada Column adalah horizontal).
FontWeight.bold	Membuat huruf tebal untuk nama.

Hasilnya, kartu profil tampil dengan avatar di kiri dan teks nama serta NIM di kanan (lihat bagian atas Gambar 1).

Bagian B. Model Data dan ListView

Pertama, data dimodelkan dengan class Makanan dan daftar konstan daftarMenu:

```
class Makanan {
  final String nama;
  final int harga;
  const Makanan(this.nama, this.harga);
}

const daftarMenu = [
  Makanan('Nasi Goreng', 15000),
  Makanan('Mie Ayam', 12000),
```

```
  Makanan('Es Teh', 4000),
  Makanan('Ayam Bakar', 20000),
];
```

Kemudian daftar ditampilkan dengan ListView.builder, Card, dan ListTile:

```
class MenuPage extends StatelessWidget {
  const MenuPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Daftar Menu')),
      body: ListView.builder(
        itemCount: daftarMenu.length,
        itemBuilder: (context, index) {
          final item = daftarMenu[index];
          return Card(
            margin: const EdgeInsets.symmetric(
                horizontal: 12, vertical: 6),
            child: ListTile(
              leading: const Icon(Icons.restaurant),
              title: Text(item.nama),
              subtitle: Text('Rp ${item.harga}'),
              trailing: const Icon(Icons.chevron_right),
            ),
          );
        },
      ),
    );
  }
}
```

Bagian Kode	Penjelasan
class Makanan	Model data yang merepresentasikan satu menu, berisi nama dan harga.
final String nama / final int harga	Properti yang nilainya tidak bisa diubah setelah objek dibuat.
const Makanan(this.nama, this.harga)	Constructor konstan. this.nama dan this.harga langsung mengisi properti dari argumen.
const daftarMenu = [...]	List berisi objek Makanan yang tidak berubah (konstan).
ListView.builder	Membuat daftar yang item-nya dibangun hanya saat akan terlihat di layar, sehingga hemat memori untuk data banyak.
itemCount	Jumlah item dalam daftar (daftarMenu.length).
itemBuilder: (context, index)	Fungsi yang dipanggil untuk setiap item; index adalah urutan item dimulai dari 0.

Bagian Kode	Penjelasan
final item = daftarMenu[index]	Mengambil objek Makanan sesuai urutan.
Card	Kartu dengan bayangan; margin mengatur jarak antar kartu.
EdgeInsets.symmetric(horizontal, vertical)	Jarak simetris: 12 di kiri-kanan dan 6 di atas-bawah.
ListTile	Baris standar dengan leading (ikon awal), title, subtitle, dan trailing (ikon akhir).
Text('Rp ${item.harga}')	String interpolation: nilai harga disisipkan ke dalam teks.

Properti home pada MyApp diubah menjadi const MenuPage(). Hasilnya, empat menu tampil sebagai kartu yang dapat di-scroll.

Bagian C. Navigasi ke Halaman Detail

Properti onTap ditambahkan pada ListTile (setelah trailing):

```
onTap: () {
  Navigator.push(
    context,
    MaterialPageRoute(
        builder: (_) => DetailPage(makanan: item)),
  );
},
```

Halaman detail menerima data lewat constructor:

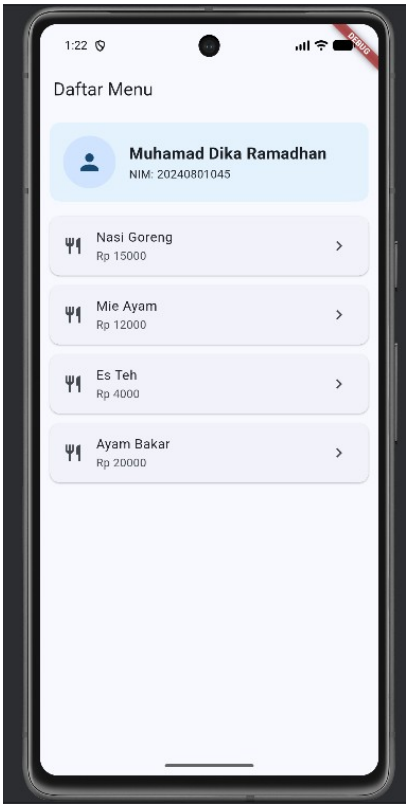
```
class DetailPage extends StatelessWidget {
  final Makanan makanan;
  const DetailPage({super.key, required this.makanan});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(makanan.nama)),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Icon(Icons.restaurant_menu, size: 80),
            const SizedBox(height: 16),
            Text(makanan.nama,
                style: const TextStyle(fontSize: 24)),
            Text('Rp ${makanan.harga}'),
            const SizedBox(height: 24),
            ElevatedButton(
              onPressed: () => Navigator.pop(context),
              child: const Text('Kembali'),
            ),
```

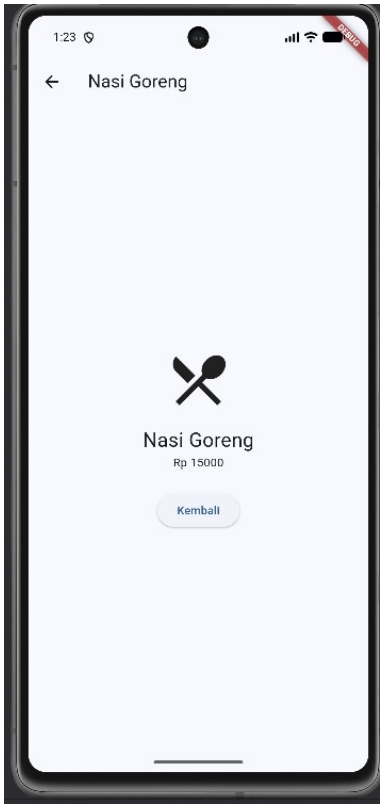
```
          ],
        ),
      ),
    );
  }
}
```

Bagian Kode	Penjelasan
onTap	Fungsi yang dijalankan ketika ListTile diketuk.
Navigator.push(context, route)	Menambahkan halaman baru di atas stack sehingga halaman tersebut tampil.
MaterialPageRoute(builder: ...)	Membuat route (rute) halaman bergaya Material, lengkap dengan animasi transisi. builder mengembalikan halaman tujuan.
(_)	Parameter context yang tidak dipakai, sehingga cukup ditulis underscore.
DetailPage(makanan: item)	Membuat halaman detail sambil mengirim objek item yang diketuk.
final Makanan makanan	Properti penampung data yang diterima DetailPage.
required this.makanan	Membuat parameter wajib diisi sehingga halaman detail tidak bisa dibuat tanpa data.
AppBar(title: Text(makanan.nama))	Judul AppBar menyesuaikan nama menu. Tombol back muncul otomatis karena halaman berada di atas stack.
Navigator.pop(context)	Menutup halaman saat ini dan kembali ke halaman sebelumnya.
ElevatedButton	Tombol Kembali yang memanggil Navigator.pop.

Hasil Praktikum Bagian A–C



Gambar 1. Kartu profil dan Daftar Menu (Bagian A–B)



Gambar 2. Halaman detail menu (Bagian C)

Pada Gambar 1 kartu profil berisi avatar, nama, dan NIM tampil di atas empat menu yang dapat di-scroll. Pada Gambar 2, setelah menu Nasi Goreng diketuk, halaman detail menampilkan data yang sesuai beserta tombol Kembali dan tombol back di AppBar.

5. Latihan Mandiri

Latihan mandiri meliputi: (1) menambah 3 menu baru, (2) menambah properti deskripsi dan menampilkannya di DetailPage, (3) mengganti Card dengan Container berwarna latar dan sudut membulat, serta (4) memformat harga menjadi format ribuan dengan fungsi buatan sendiri.

Model data dan fungsi format harga

```
class Makanan {
  final String nama;
  final int harga;
  final String deskripsi;
  const Makanan(this.nama, this.harga, this.deskripsi);
}

// fungsi buatan sendiri: 15000 -> 15.000
String formatRupiah(int angka) {
  final s = angka.toString();
  final buf = StringBuffer();
```

```
  for (int i = 0; i < s.length; i++) {
    if (i > 0 && (s.length - i) % 3 == 0) buf.write('.');
    buf.write(s[i]);
  }
  return 'Rp $buf';
}

const daftarMenu = [
  Makanan('Nasi Goreng', 15000, '...'),
  Makanan('Mie Ayam', 12000,
      'Mie kenyal dengan toping ayam manis gurih.'),
  Makanan('Es Teh', 4000, '...'),
  Makanan('Ayam Bakar', 20000, '...'),
  Makanan('Soto Ayam', 18000, '...'),   // menu baru
  Makanan('Bakso Sapi', 16000, '...'),  // menu baru
  Makanan('Es Jeruk', 5000, '...'),     // menu baru
];
```

Container pengganti Card pada daftar

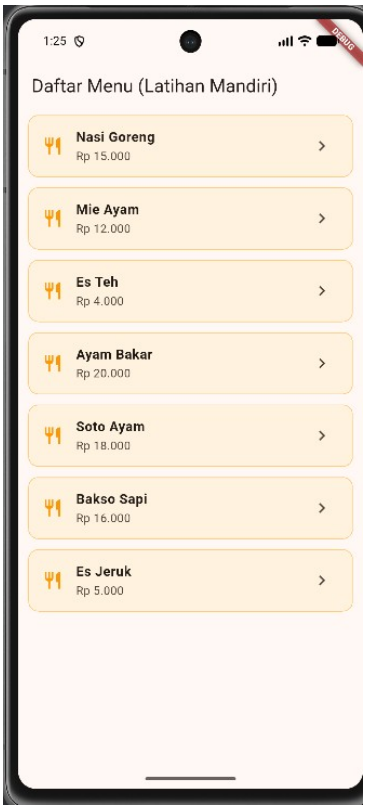
```
return Container(
  margin: const EdgeInsets.symmetric(
      horizontal: 12, vertical: 6),
  decoration: BoxDecoration(
    color: Colors.orange.shade50,
    borderRadius: BorderRadius.circular(12),
    border: Border.all(color: Colors.orange.shade200),
  ),
  child: ListTile(
    leading: const Icon(Icons.restaurant,
        color: Colors.orange),
    title: Text(item.nama,
        style: const TextStyle(
            fontWeight: FontWeight.bold)),
    subtitle: Text(formatRupiah(item.harga)),
    trailing: const Icon(Icons.chevron_right),
    onTap: () => Navigator.push(context,
        MaterialPageRoute(
            builder: (_) => DetailPage(makanan: item))),
  ),
);
```

Tampilan deskripsi di DetailPage

```
// di dalam Column pada DetailPage
Text(makanan.nama,
    style: const TextStyle(fontSize: 24,
        fontWeight: FontWeight.bold)),
Text(formatRupiah(makanan.harga),
    style: const TextStyle(color: Colors.grey)),
const SizedBox(height: 16),
Padding(
```

```
  padding: const EdgeInsets.symmetric(horizontal: 24),
  child: Text(makanan.deskripsi,
      textAlign: TextAlign.center),
),
```

Bagian Kode	Penjelasan
final String deskripsi	Properti baru pada class Makanan; constructor diperbarui menerima tiga argumen.
formatRupiah(int angka)	Fungsi buatan sendiri yang mengubah angka menjadi teks berformat ribuan, misalnya 15000 menjadi Rp 15.000.
angka.toString()	Mengubah angka menjadi String agar bisa diproses per karakter.
StringBuffer	Penampung teks yang efisien untuk menyusun string sedikit demi sedikit; write() menambahkan karakter.
(s.length - i) % 3 == 0	Operator modulo untuk mendeteksi posisi setiap kelipatan tiga digit dari kanan, tempat titik pemisah disisipkan.
Makanan(...) tambahan	Tiga menu baru (Soto Ayam, Bakso Sapi, Es Jeruk) membuat daftar menjadi tujuh item dan tetap dapat di-scroll berkat ListView.builder.
Container + BoxDecoration	Pengganti Card: warna latar oranye muda (shade50), sudut membulat (BorderRadius), dan garis tepi (Border.all).
Colors.orange.shade200	Varian oranye lebih tua yang dipakai untuk garis tepi.
textAlign: TextAlign.center	Meratakan teks deskripsi ke tengah.



Gambar 3. Daftar menu memakai Container, 7 menu, harga berformat ribuan



Gambar 4. Halaman detail dengan deskripsi dan harga berformat

Pada Gambar 3 daftar kini berisi tujuh menu dengan kartu Container berlatar oranye muda dan sudut membulat, serta harga tampil dalam format ribuan (Rp 15.000). Pada Gambar 4 halaman detail menampilkan deskripsi menu di bawah harga.

6. Tugas: Aplikasi Daftar Kontak

Aplikasi Daftar Kontak berisi enam kontak (nama, nomor telepon, email) yang disimpan dalam list berisi objek class. Halaman utama menampilkan daftar dengan ListView.builder dan ListTile beravatar huruf pertama nama, dan mengetuk kontak membuka halaman detail.

Model data

```
class Kontak {
  final String nama;
  final String telepon;
  final String email;
  const Kontak(this.nama, this.telepon, this.email);
}

const daftarKontak = [
  Kontak('Muhamad Dika Ramadhan', '0812352260', '(email)'),
  Kontak('Budi Santoso', '082198765432', '(email)'),
  Kontak('Citra Dewi', '083811223344', '(email)'),
  Kontak('Deni Kurniawan', '085755667788', '(email)'),
```

```
  Kontak('Eka Putra', '087899001122', '(email)'),
  Kontak('Fani Rahmawati', '089633445566', '(email)'),
];
```

Halaman daftar kontak

```
class KontakPage extends StatelessWidget {
  const KontakPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Daftar Kontak')),
      body: ListView.builder(
        itemCount: daftarKontak.length,
        itemBuilder: (context, index) {
          final k = daftarKontak[index];
          return Card(
            margin: const EdgeInsets.symmetric(
                horizontal: 12, vertical: 6),
            child: ListTile(
              leading: CircleAvatar(
                child: Text(k.nama[0]),   // huruf pertama
              ),
              title: Text(k.nama,
                  style: const TextStyle(
                      fontWeight: FontWeight.bold)),
              subtitle: Text(k.telepon),
              trailing: const Icon(Icons.chevron_right),
              onTap: () => Navigator.push(
                context,
                MaterialPageRoute(
                    builder: (_) => DetailKontak(kontak: k)),
              ),
            ),
          );
        },
      ),
    );
  }
}
```

Halaman detail kontak

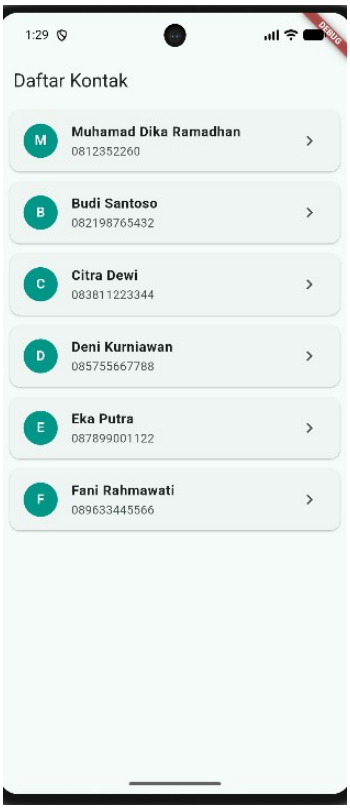
```
class DetailKontak extends StatelessWidget {
  final Kontak kontak;
  const DetailKontak({super.key, required this.kontak});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Detail Kontak')),
```

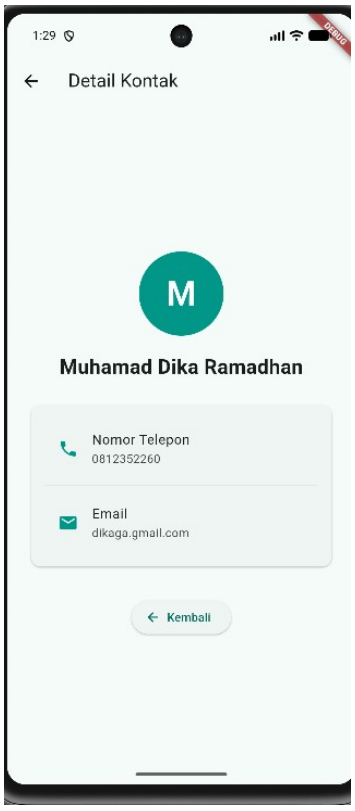
```
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            CircleAvatar(
              radius: 48,
              child: Text(kontak.nama[0],
                  style: const TextStyle(fontSize: 40)),
            ),
            const SizedBox(height: 16),
            Text(kontak.nama,
                style: const TextStyle(fontSize: 22,
                    fontWeight: FontWeight.bold)),
            const SizedBox(height: 24),
            Card(
              margin: const EdgeInsets.symmetric(
                  horizontal: 24),
              child: Column(
                children: [
                  ListTile(
                    leading: const Icon(Icons.phone),
                    title: const Text('Nomor Telepon'),
                    subtitle: Text(kontak.telepon),
                  ),
                  const Divider(height: 1),
                  ListTile(
                    leading: const Icon(Icons.email),
                    title: const Text('Email'),
                    subtitle: Text(kontak.email),
                  ),
                ],
              ),
            ),
            const SizedBox(height: 24),
            ElevatedButton.icon(
              onPressed: () => Navigator.pop(context),
              icon: const Icon(Icons.arrow_back),
              label: const Text('Kembali'),
            ),
          ],
        ),
      ),
    );
  }
}
```

Bagian Kode	Penjelasan
class Kontak	Model data satu kontak: nama, telepon, dan email.
daftarKontak	List berisi enam objek Kontak sebagai sumber data.
KontakPage	Halaman utama yang menampilkan seluruh kontak.

Bagian Kode	Penjelasan
ListView.builder + Card + ListTile	Menampilkan daftar kontak yang dapat di-scroll, satu kartu per kontak.
CircleAvatar(child: Text(k.nama[0]))	Avatar bulat berisi huruf pertama nama. Indeks [0] mengambil karakter pertama dari String.
subtitle: Text(k.telepon)	Menampilkan nomor telepon di bawah nama.
onTap + Navigator.push	Membuka DetailKontak sambil mengirim objek kontak yang diketuk lewat constructor.
DetailKontak	Halaman detail yang menerima satu objek Kontak (final Kontak kontak) dan menampilkan seluruh datanya.
Card berisi dua ListTile	Menampilkan nomor telepon (ikon phone) dan email (ikon email) dalam satu kartu.
Divider(height: 1)	Garis pemisah tipis antara baris telepon dan email.
ElevatedButton.icon	Tombol dengan ikon dan label. onPressed memanggil Navigator.pop(context) untuk kembali.



Gambar 5. Halaman daftar kontak



Gambar 6. Halaman detail kontak

Gambar 5 menampilkan enam kontak dengan avatar berisi huruf pertama nama. Gambar 6 menampilkan detail kontak yang dipilih lengkap dengan nomor telepon, email, dan tombol Kembali.

7. Jawaban Pertanyaan Refleksi

1. Apa perbedaan ListView biasa dengan ListView.builder?

ListView biasa membuat seluruh item sekaligus dari daftar children yang ditulis manual, sehingga cocok untuk jumlah item sedikit dan tetap. ListView.builder membuat item hanya ketika akan tampil di layar (lazy) berdasarkan itemCount dan itemBuilder, sehingga jauh lebih hemat memori dan efisien untuk data yang banyak atau berasal dari list.

2. Mengapa Row yang berisi teks panjang dapat menyebabkan overflow, dan bagaimana Expanded membantu?

Row tidak membatasi lebar anak-anaknya pada sumbu utama, sehingga teks panjang akan terus melebar melewati batas layar dan muncul garis kuning-hitam (overflow). Expanded memaksa anak hanya mengisi sisa ruang yang tersedia, sehingga lebar teks dibatasi dan teks otomatis turun ke baris berikutnya (wrap).

3. Bagaimana data dikirim dari halaman daftar ke halaman detail pada praktikum ini?

Data dikirim lewat constructor. Saat item diketuk, onTap memanggil Navigator.push dengan MaterialPageRoute yang membuat halaman detail sambil menyertakan objek yang dipilih, misalnya DetailPage(makanan: item). Halaman detail menyimpannya di properti final dan menampilkannya, dengan parameter yang ditandai required agar wajib diisi.

8. Kesimpulan

Dari praktikum ini dapat disimpulkan bahwa layout Flutter disusun dari kombinasi Container, Padding, Row, Column, dan Expanded yang masing-masing punya peran jelas. Data dimodelkan dengan class Dart lalu ditampilkan secara efisien memakai ListView.builder, Card, dan ListTile. Perpindahan halaman dikelola Navigator dengan push untuk membuka dan pop untuk menutup, sedangkan data dikirim melalui constructor halaman tujuan. Konsep ini dipakai untuk membuat aplikasi Daftar Menu dan Daftar Kontak yang interaktif.